

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КРЕМЕНЧУЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ МИХАЙЛА ОСТРОГРАДСЬКОГО
НАВЧАЛЬНО-НАУКОВИЙ ІНСТИТУТ ЕЛЕКТРИЧНОЇ ІНЖЕНЕРІЇ
ТА ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

Кафедра автоматизації та інформаційних систем

ОСВІТНІЙ КОМПОНЕНТ
«ОСНОВИ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ»

ЗВІТ
З ЛАБОРАТОРНОЇ РОБОТИ № 7

Виконав:

студент групи кн-25-1

Рафієв Б. С.

Перевірила:

доцент кафедри АІС

Істоміна Н.М.

Кременчук 2025

Тема: Работа з Git та GitHub

Мета: ознайомитися з принципами роботи системи контролю версій Git та платформою GitHub; навчитися виконувати базові операції в Git Bash і VS Code.

Порядок виконання роботи

1. Работа з Git Bash.
 - запустити Git Bash на диску D;

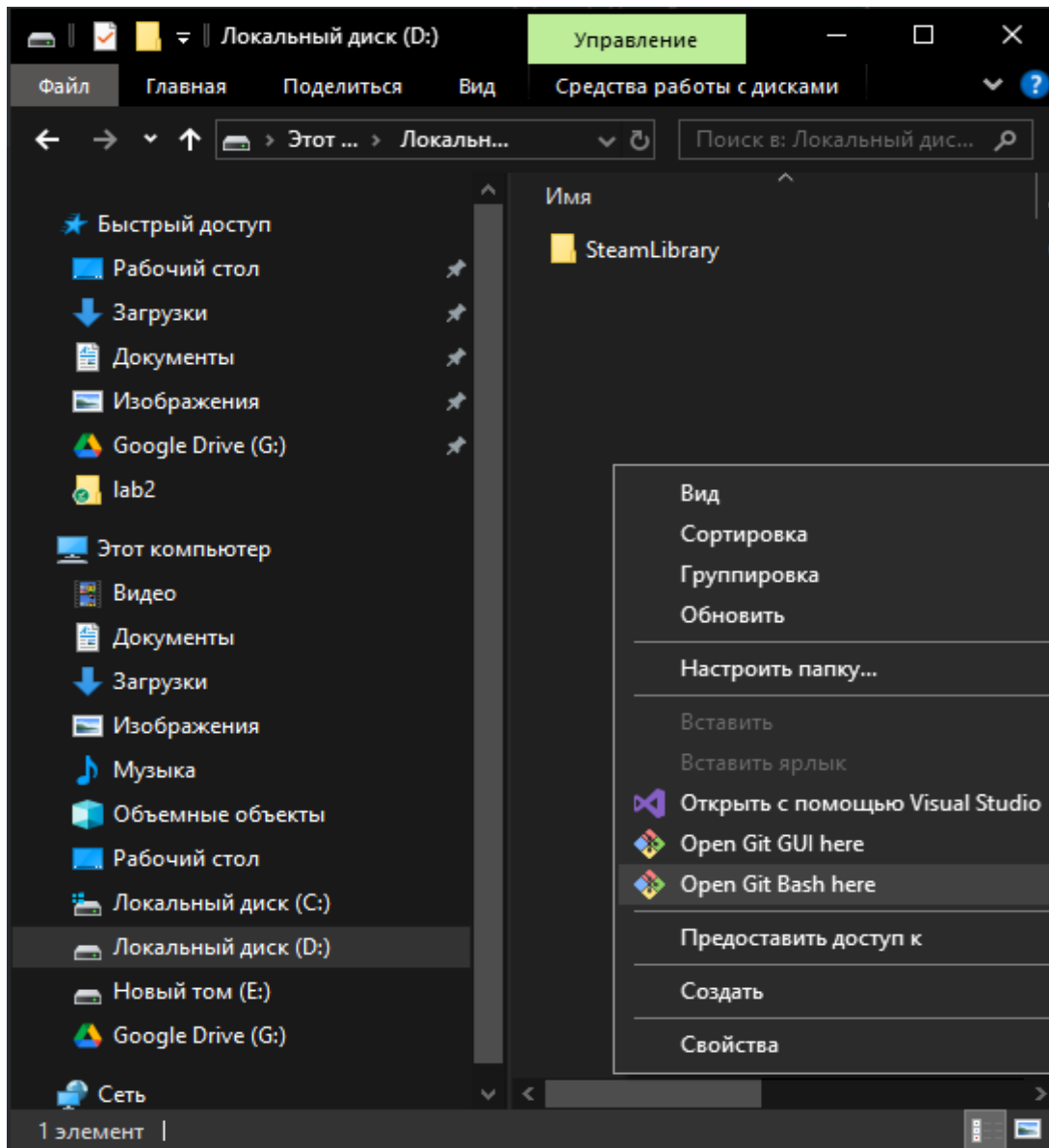
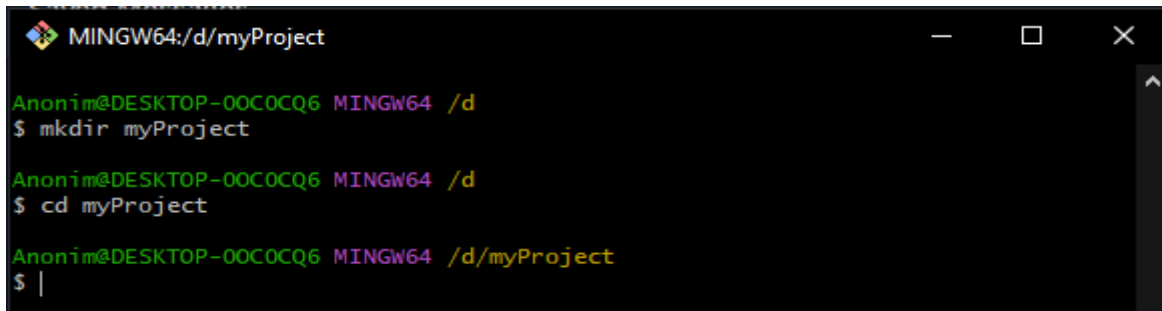


Рисунок 1 – запуск Git Bash

- створити каталог «myProject»;



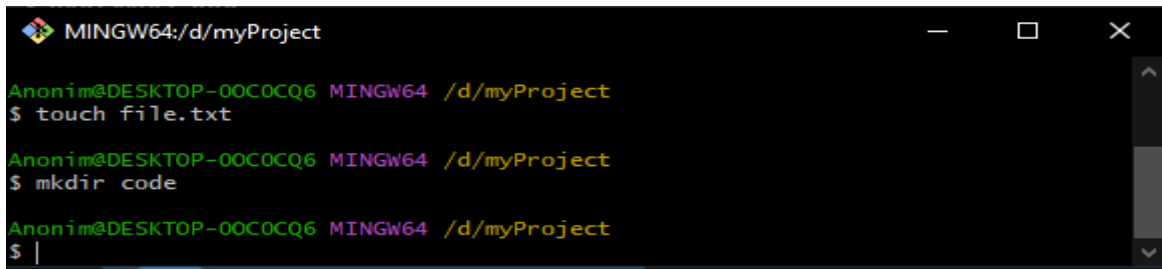
```
MINGW64:/d/myProject
Anonim@DESKTOP-00C0CQ6 MINGW64 /d
$ mkdir myProject

Anonim@DESKTOP-00C0CQ6 MINGW64 /d
$ cd myProject

Anonim@DESKTOP-00C0CQ6 MINGW64 /d/myProject
$ |
```

Рисунок 2 – створення каталогу «myProject»

- в середині проєкту створити файл з розширенням «txt» та каталог «code»;



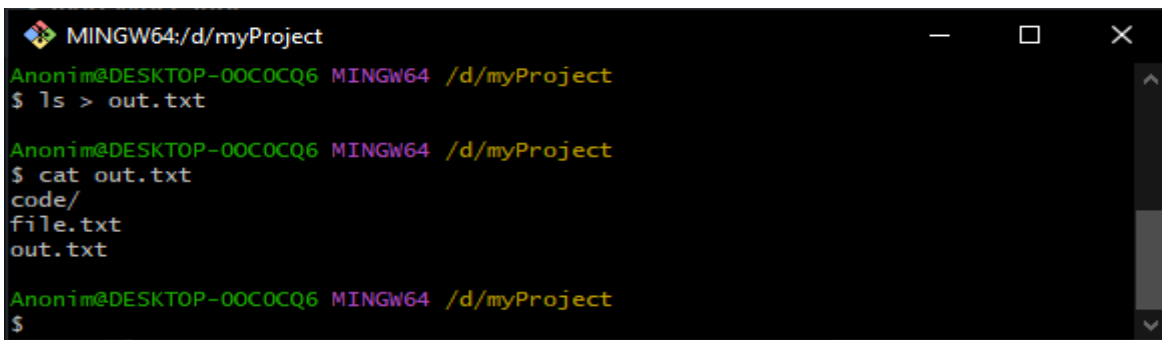
```
MINGW64:/d/myProject
Anonim@DESKTOP-00C0CQ6 MINGW64 /d/myProject
$ touch file.txt

Anonim@DESKTOP-00C0CQ6 MINGW64 /d/myProject
$ mkdir code

Anonim@DESKTOP-00C0CQ6 MINGW64 /d/myProject
$ |
```

Рисунок 3 – створення файлу з розширенням «txt» та каталогу «code»

- переглянути вміст каталогу проєкту і записати результат у файл «out.txt»;



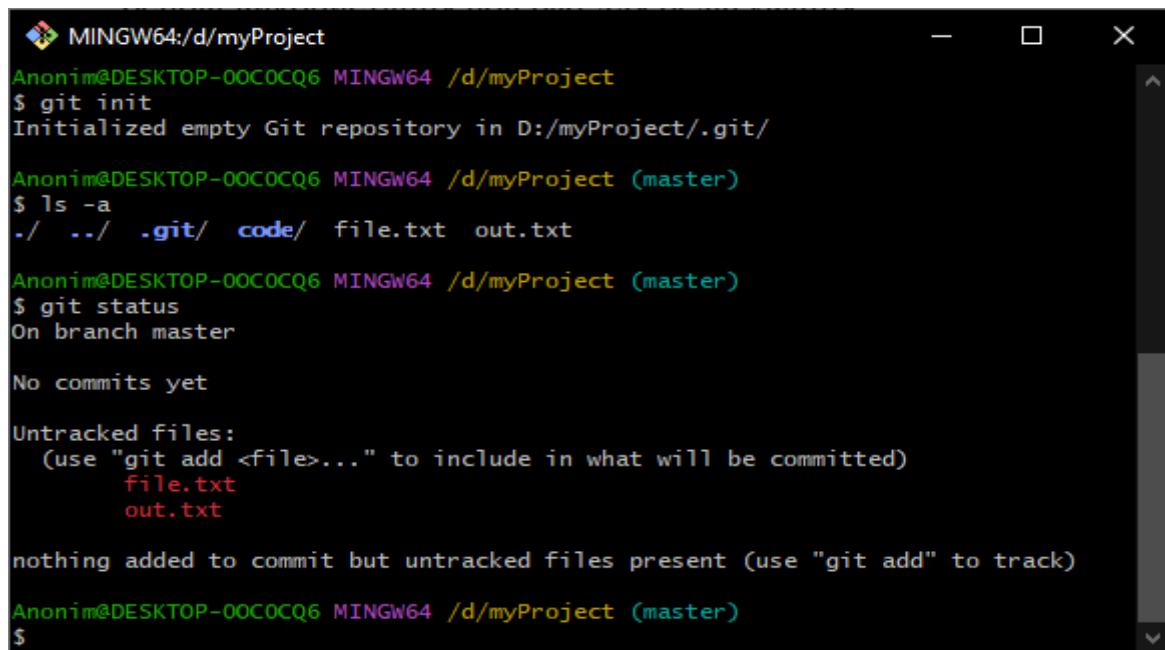
```
MINGW64:/d/myProject
Anonim@DESKTOP-00C0CQ6 MINGW64 /d/myProject
$ ls > out.txt

Anonim@DESKTOP-00C0CQ6 MINGW64 /d/myProject
$ cat out.txt
code/
file.txt
out.txt

Anonim@DESKTOP-00C0CQ6 MINGW64 /d/myProject
$
```

Рисунок 4 – перегляд вмісту каталогу проєкту і запис результату у файл «out.txt»

- підключити до Вашого проєкту Git (ініціювати), переглянути вміст директорії проєкту (ls -a), переконатись у наявності прихованого каталогу «.git», перевірити статус Git;

A screenshot of a terminal window titled 'MINGW64:/d/myProject'. The window shows the following commands and output:

```
Anonim@DESKTOP-00C0CQ6 MINGW64 /d/myProject
$ git init
Initialized empty Git repository in D:/myProject/.git/

Anonim@DESKTOP-00C0CQ6 MINGW64 /d/myProject (master)
$ ls -a
./ ../ .git/ code/ file.txt out.txt

Anonim@DESKTOP-00C0CQ6 MINGW64 /d/myProject (master)
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    file.txt
    out.txt

nothing added to commit but untracked files present (use "git add" to track)

Anonim@DESKTOP-00C0CQ6 MINGW64 /d/myProject (master)
$
```

Рисунок 5 –

- додати робочий каталог до Git: виконати послідовно додавання всього вмісту проєкту та коміт, між операціями перевіряти статус Git;
- відредагувати вміст текстового файлу за допомогою GNU nano, додати такий текстовий рядок: `print("First try")`, зберегти та вийти з редактору GNU nano, перевірити статус Git (Ви повинні бачити статус для цього файлу «modified», додайте змінений файл та виконайте коміт;

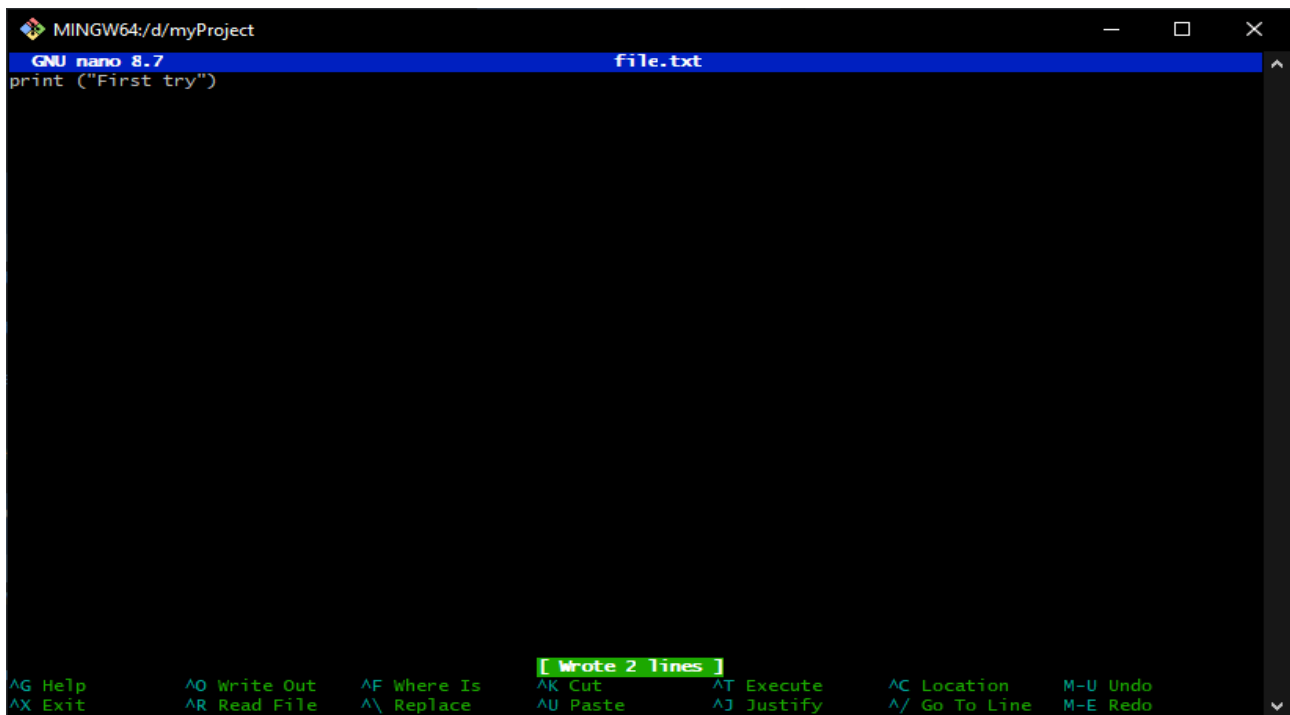


Рисунок 6 – редагування вмісту текстового файлу за допомогою GNU nano,
додавання текстового рядка

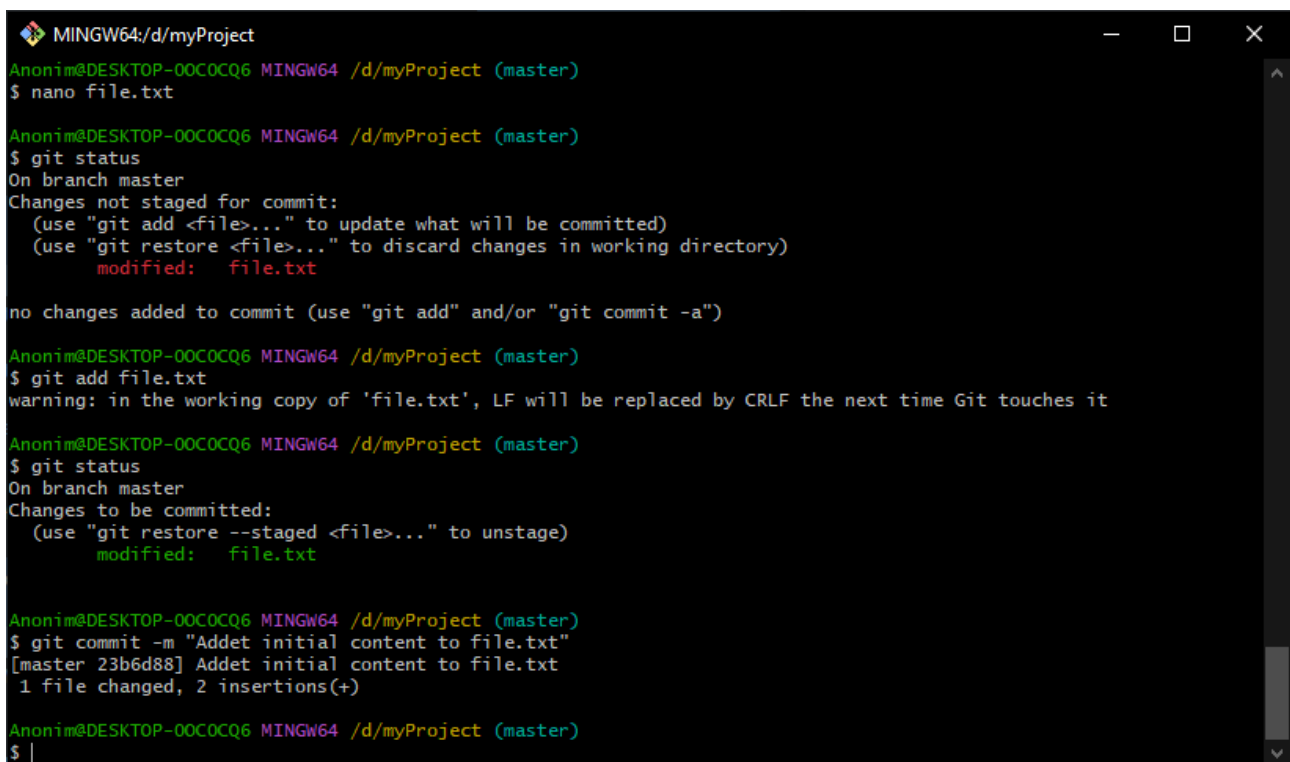


Рисунок 7 – перевірка статусу Git

- створити нову гілку, створити файл «hello_world.txt», відредагувати за допомогою notepad і додати такий текстовий рядок: `print("Hello world :)")`, перевірити статус, додати та зробити комміт;

```

MINGW64:/d/myProject
Anonim@DESKTOP-00C0CQ6 MINGW64 /d/myProject (master)
$ git checkout -b feature-hello
Switched to a new branch 'feature-hello'

Anonim@DESKTOP-00C0CQ6 MINGW64 /d/myProject (feature-hello)
$ touch hello_world.txt

Anonim@DESKTOP-00C0CQ6 MINGW64 /d/myProject (feature-hello)
$ notepad hello_world.txt

Anonim@DESKTOP-00C0CQ6 MINGW64 /d/myProject (feature-hello)
$ git status
On branch feature-hello
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    hello_world.txt

nothing added to commit but untracked files present (use "git add" to track)

Anonim@DESKTOP-00C0CQ6 MINGW64 /d/myProject (feature-hello)
$ git add hello)world.txt
bash: syntax error near unexpected token `)'

Anonim@DESKTOP-00C0CQ6 MINGW64 /d/myProject (feature-hello)
$ git add hello_world.txt

Anonim@DESKTOP-00C0CQ6 MINGW64 /d/myProject (feature-hello)
$ git commit -m "Addet initial content to file.txt"hello_world.txt in feature-hello branch"
> "
error: pathspec 'in' did not match any file(s) known to git
error: pathspec 'feature-hello' did not match any file(s) known to git
error: pathspec 'branch
' did not match any file(s) known to git

Anonim@DESKTOP-00C0CQ6 MINGW64 /d/myProject (feature-hello)
$ git commit -m "Added hello_world.txt in feature-hello branch"
[feature-hello abc6e57] Added hello_world.txt in feature-hello branch
1 file changed, 1 insertion(+)
create mode 100644 hello_world.txt

Anonim@DESKTOP-00C0CQ6 MINGW64 /d/myProject (feature-hello)
$

```

- Рисунок 8 – створення нової гілки, створення файлу, редагування за допомогою notepad і додавання текстового рядка, перевірка статусу
- переглянути коротку та детальну історію коммітів;

```
MINGW64:/d/myProject
Anonim@DESKTOP-00C0CQ6 MINGW64 /d/myProject (feature-hello)
$ git log --oneline
abc6e57 (HEAD -> feature-hello) Added hello_world.txt in feature-hello branch
23b6d88 (master) Addet initial content to file.txt
4810fff Initial project setup with file.txt, out.txt, and code directory

Anonim@DESKTOP-00C0CQ6 MINGW64 /d/myProject (feature-hello)
$ git log
commit abc6e573decfed194f4b8a8f592edc5c1c80d89 (HEAD -> feature-hello)
Author: Qnonimbba <borayon@gmail.com>
Date: Tue Dec 2 00:52:57 2025 +0200

    Added hello_world.txt in feature-hello branch

commit 23b6d880fbfd6fc349707cc40da31f3d469f5ae8 (master)
Author: Qnonimbba <borayon@gmail.com>
Date: Tue Dec 2 00:47:44 2025 +0200

    Addet initial content to file.txt

commit 4810fffa0684bc258d99c4318fdb93ee38a76a7
Author: Qnonimbba <borayon@gmail.com>
Date: Tue Dec 2 00:39:21 2025 +0200

    Initial project setup with file.txt, out.txt, and code directory

Anonim@DESKTOP-00C0CQ6 MINGW64 /d/myProject (feature-hello)
$ |
```

Рисунок 9 – перегляд короткої та детальної історії комітів

- виконати злиття гілок.

```
MINGW64:/d/myProject
Anonim@DESKTOP-00C0CQ6 MINGW64 /d/myProject (feature-hello)
$ git checkout master
Switched to branch 'master'

Anonim@DESKTOP-00C0CQ6 MINGW64 /d/myProject (master)
$ git merge feature-hello
Updating 23b6d88..abc6e57
Fast-forward
 hello_world.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 hello_world.txt

Anonim@DESKTOP-00C0CQ6 MINGW64 /d/myProject (master)
$ git log --oneline
abc6e57 (HEAD -> master, feature-hello) Added hello_world.txt in feature-hello branch
23b6d88 Addet initial content to file.txt
4810fff Initial project setup with file.txt, out.txt, and code directory

Anonim@DESKTOP-00C0CQ6 MINGW64 /d/myProject (master)
$ |
```

Рисунок 10 – виконання злиття гілок

2. Робота з Git у VS Code:
- створити копію каталогу «myProject», видалити з нього Git;

```
MINGW64/d/myProject_vs
Anonim@DESKTOP-00C0CQ6 MINGW64 /d/myProject (master)
$ cd /d/

Anonim@DESKTOP-00C0CQ6 MINGW64 /d
$ cp -r myProject myProject_vs

Anonim@DESKTOP-00C0CQ6 MINGW64 /d
$ cd myProject_vs

Anonim@DESKTOP-00C0CQ6 MINGW64 /d/myProject_vs (master)
$ rm -rf .git

Anonim@DESKTOP-00C0CQ6 MINGW64 /d/myProject_vs
$ |
```

Рисунок 11 – створення копії каталогу «myProject», видалення з нього Git

- відкрити цей проєкт у VS Code, ініціювати Git;
- створити «.gitignore», прописати ігнорування логів;

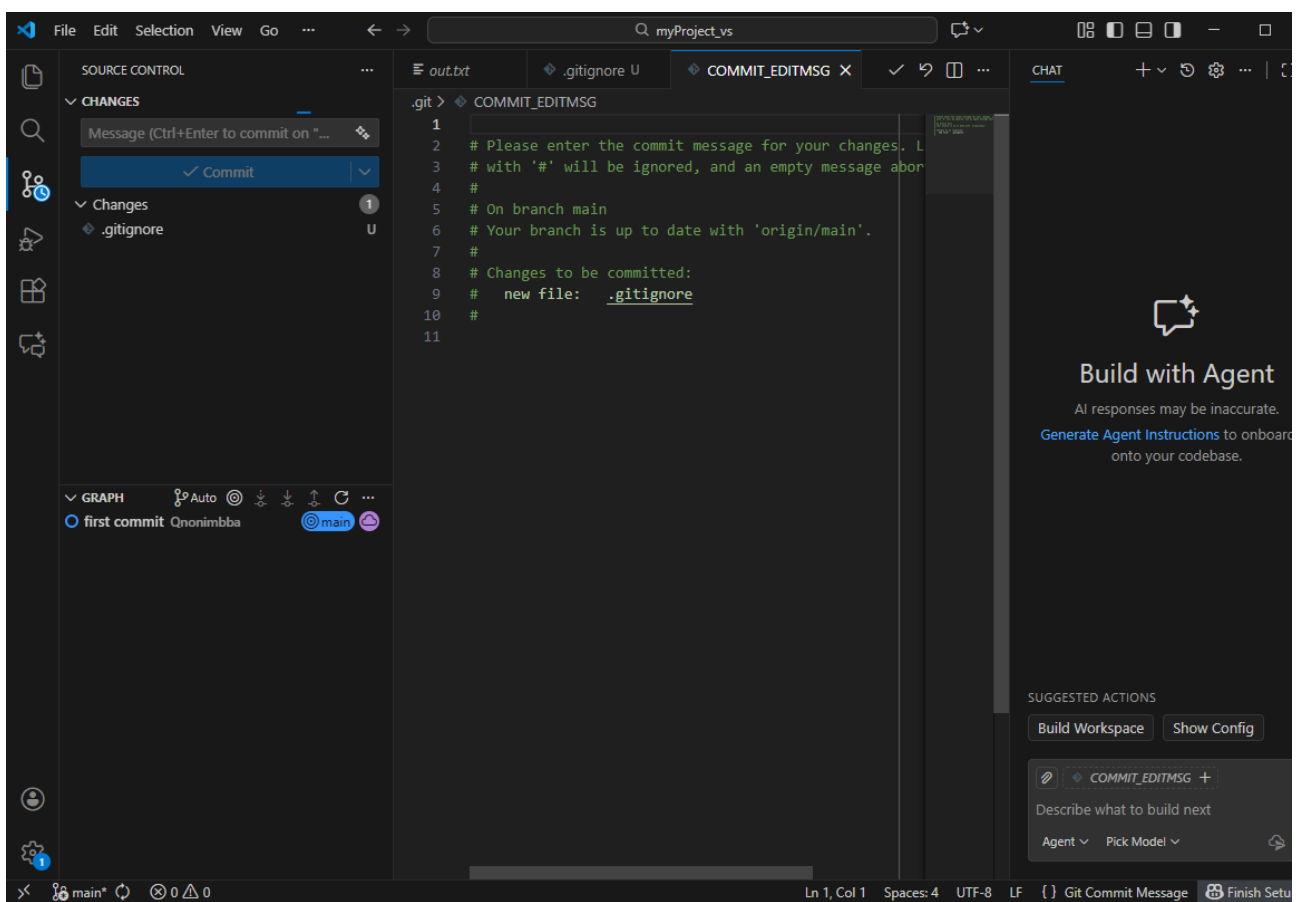


Рисунок 12 – створення «.gitignore»

- зробити перший коміт проєкту;
- створити нову гілку;
- у новій гілці змінити текстовий файл, виконати додавання та коміт;

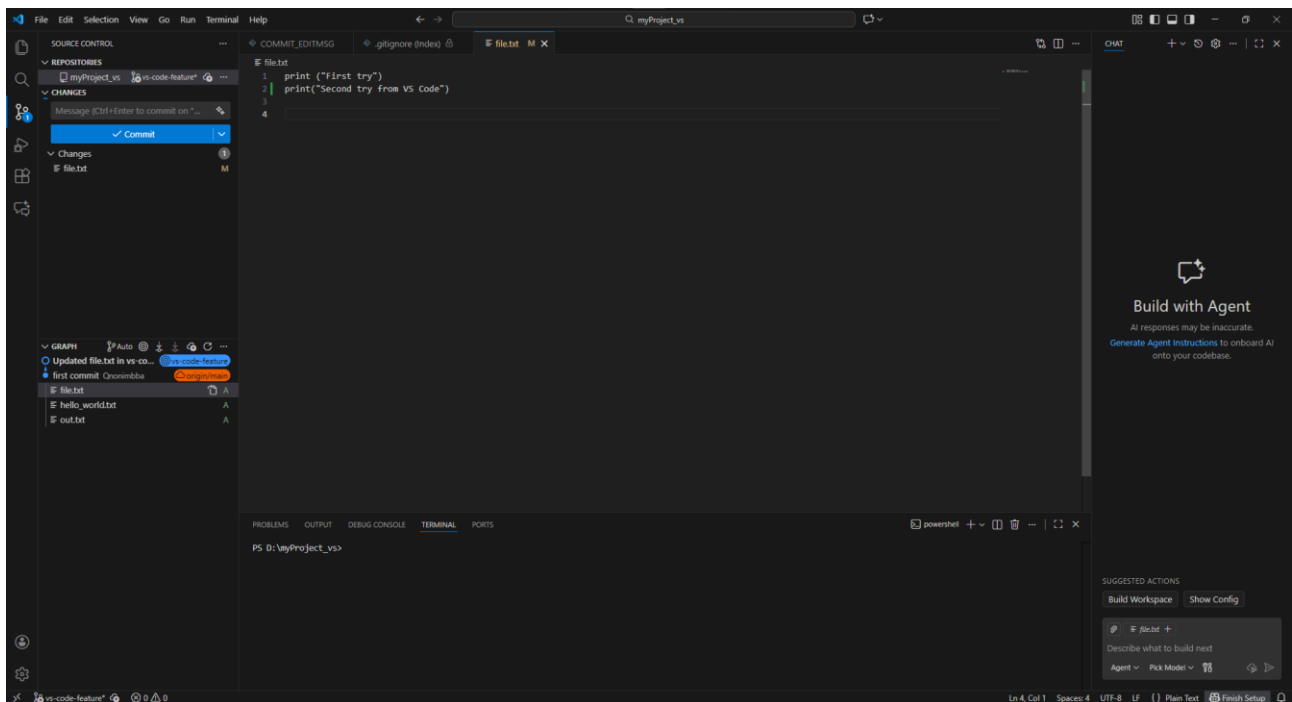


Рисунок 13 – зміна текстового файлу у новій гілці, виконання додавання та КОМІТ

- зЛИТИ з ОСНОВНОЮ ГІЛКОЮ;
- відправити на GitHub.

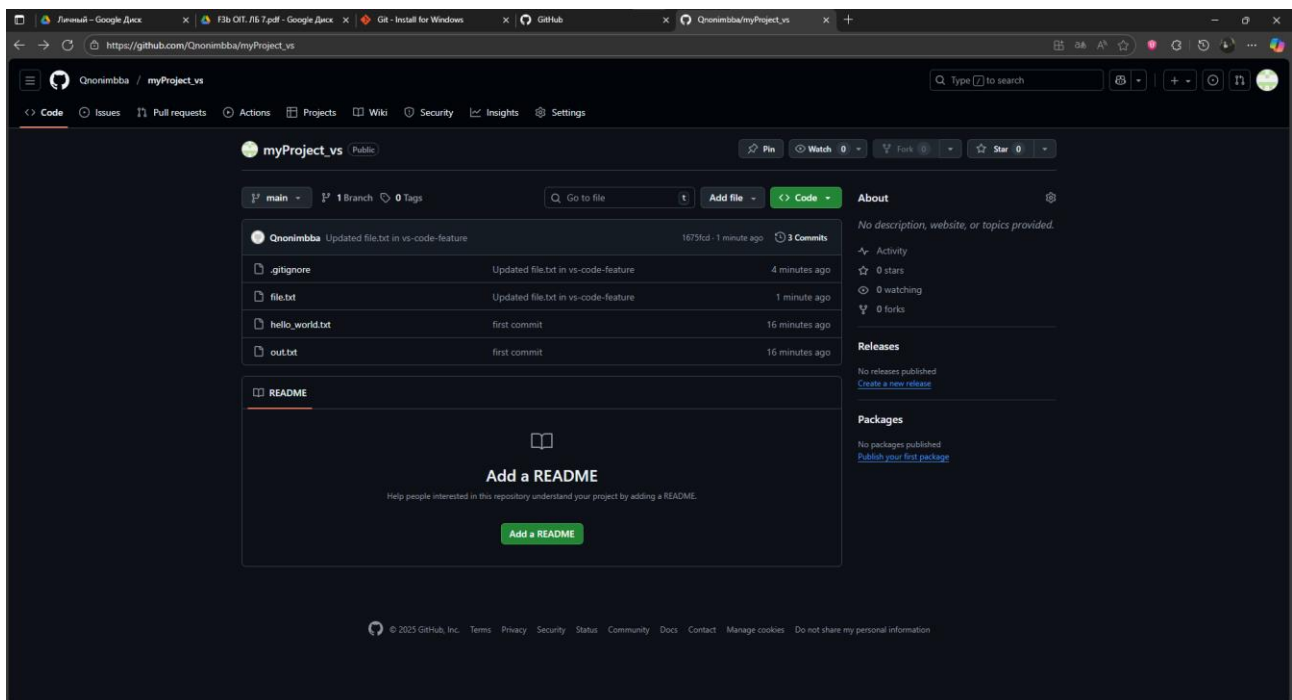


Рисунок 14 – перегляд на GitHub.

3. Робота з GitHub:

- зареєструватись на GitHub через свій gmail;
- зайти у свій профіль, перевірити свої репозиторії;

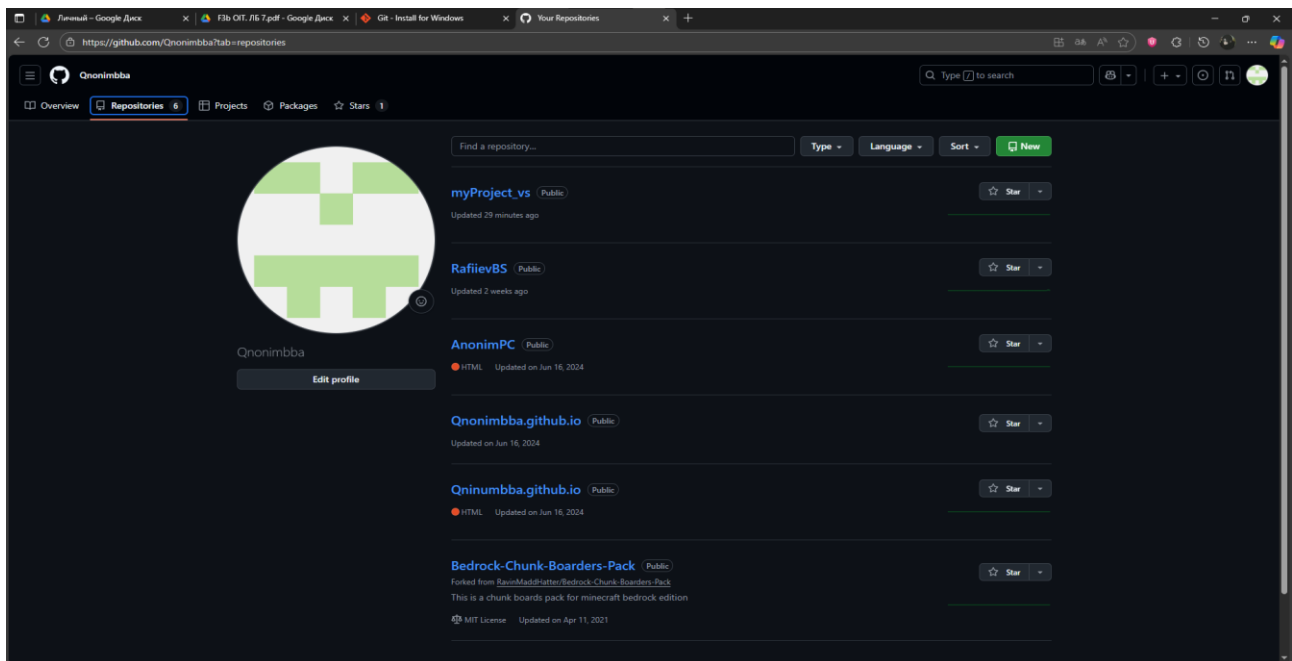


Рисунок 15 – перегляд репозиторієв на GitHub.

- створити репозиторій, якій за ім'ям співпадає з іменем вашого акаунту;
- відредагувати файл README.md та створити опис власного профілю

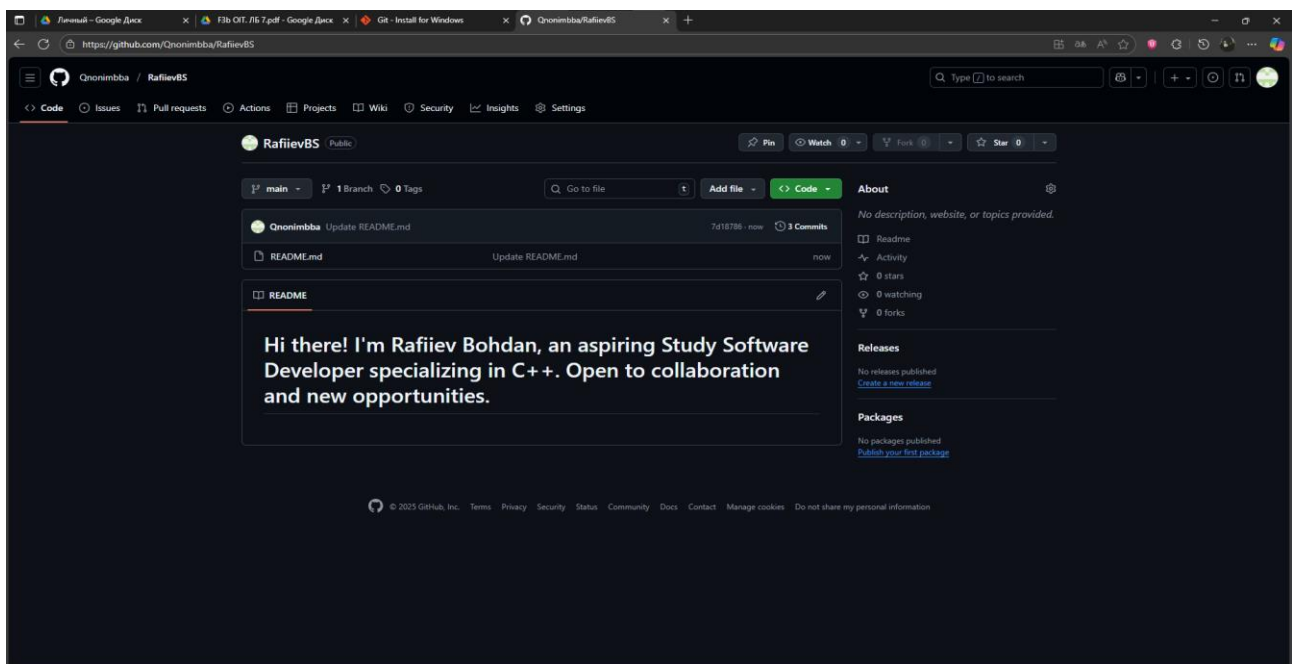


Рисунок 16 – створення репозиторію, редагування README.md та створення опису власного профілю

Висновки: Завдяки цій лабораторній роботі я ознайомився з принципами роботи системи контролю версій Git та платформою GitHub; навчився виконувати базові операції в Git Bash і VS Code.